1. 왜 쿠버네티스인가 - 서버에서 컨테이너로

배포 단위의 변화: 물리 서버 → VM → 컨테이너 → 컨테이너 오케스트레이션

단계	배포 단위	남는 문제
물리 서버	서버 1대 = 앱 1개	자원 낭비, 증설에 수 주 소요
VM	VM 1대 = OS + 앱	VM마다 OS가 있어 무겁고, 서버별 설정이 조금씩 달라짐(구성 드리프트)
컨테이너	이미지 1개 = 앱 + 라이브러리	컨테이너가 수백 개가 되면 배치·재시작·연결을 사람이 관리할 수 없음
쿠버네티스	"이렇게 유지해 줘"라는 선언	배치·복구·확장·연결을 시스템이 대신 수행

비유 - 밀키트와 조리대

- 컨테이너 이미지 = 레시피와 재료가 한 봉지에 든 표준 밀키트. 어느 매장에서 뜯어도 같은 맛 → "내 PC에서는 됐는데" 문제 제거(이식성)
- 컨테이너 = 그 밀키트를 뜯어 조리 중인 조리대. VM이 "매장을 통째로 하나 더 짓는 것"이라면 컨테이너는 "조리대 하나 추가"
- 매장이 수십 곳, 조리대가 수백 개가 되면 누가 어디서 무엇을 만들지 정해 주는 본사가 필요 → 이 본사 역할이 쿠버네티스

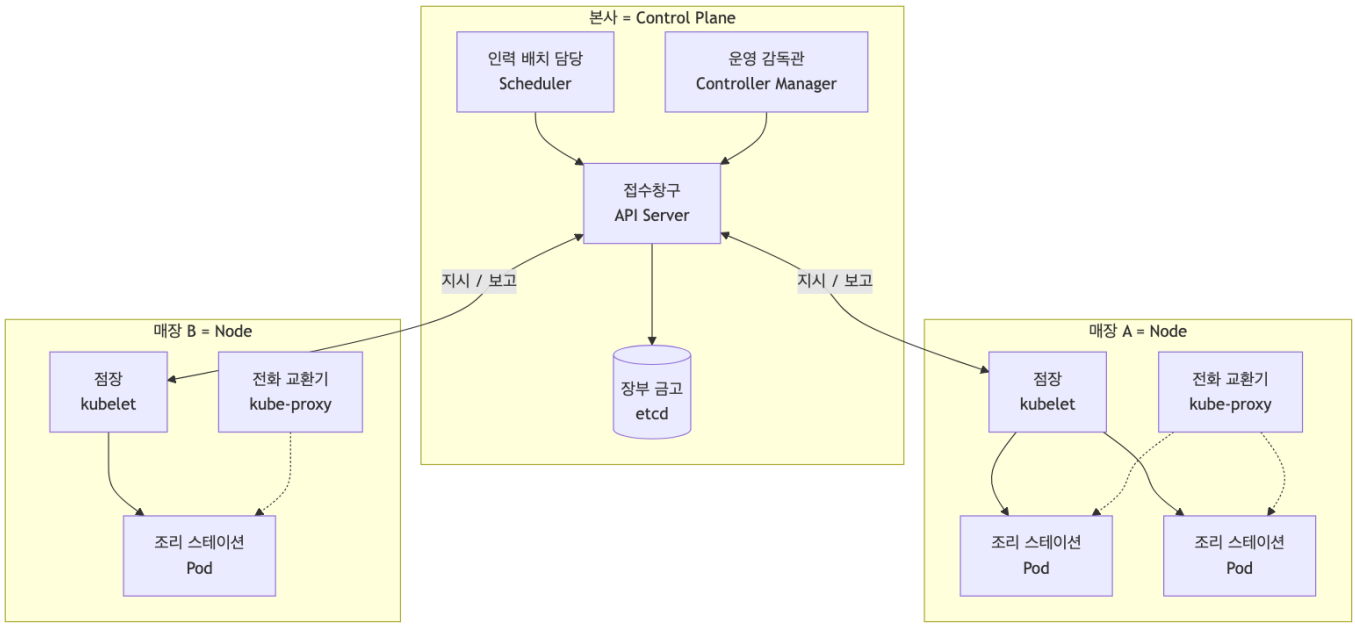
쿠버네티스가 대신 해 주는 일 (7월 교안 기능 목록 기준)

- 서비스 디스커버리·로드밸런싱: 컨테이너 IP가 바뀌어도 이름으로 연결
- 스토리지 오케스트레이션: 컨테이너가 다른 서버로 옮겨 가도 같은 볼륨 연결
- 자동 롤아웃·롤백: 버전 교체를 단계적으로, 문제 시 이전 버전으로
- 자동 빈패킹: 서버별 여유 자원에 맞춰 컨테이너 배치
- 자동 복구(Self-healing): 죽은 컨테이너 재시작, 서버 장애 시 다른 서버로 재배치
- 시크릿·구성 관리: 이미지를 다시 만들지 않고 설정·비밀번호 교체

> **한 줄 정의**
>
> - 쿠버네티스 = 컨테이너에 대한 원하는 상태(Desired State)를 받아 현재 상태를 계속 그쪽으로 맞추는 오픈소스 컨테이너 오케스트레이션 플랫폼
> - 이름은 그리스어 "조타수", 약칭 K8s(K와 s 사이 8글자). 구글 내부 시스템(Borg)에서 출발해 CNCF가 관리
> - 1~12장: 일반 쿠버네티스 개념(외부 지식 보강) / 13~20장: SCP Hands-on Labs 1·2와 SCP 매뉴얼 기준 SCP 적용

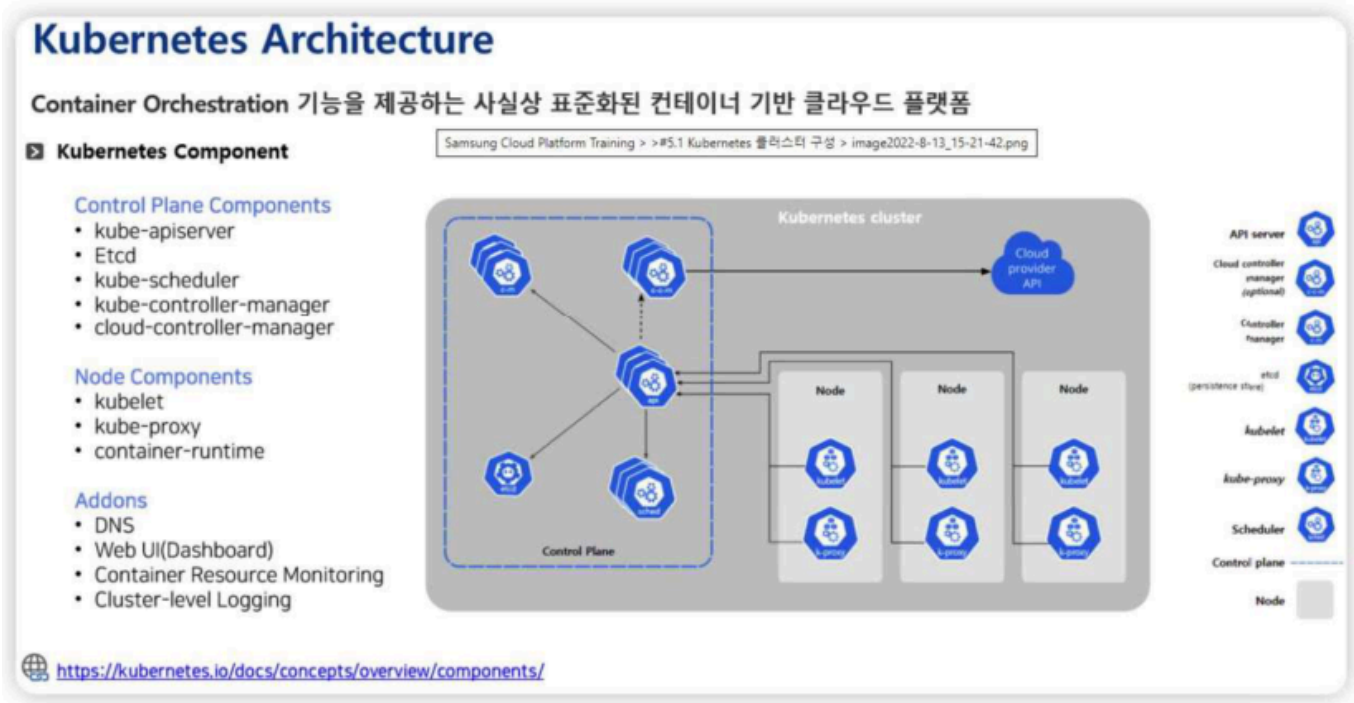
2. 비유 지도 - 치킨 프랜차이즈 하나로 전체 보기

클러스터 전체를 하나의 치킨 프랜차이즈로 놓고 본 대응표. 3~12장은 이 표의 한 줄씩을 풀어 쓴 내용



구분	쿠버네티스	프랜차이즈 비유	한 줄 역할
전체	Cluster	프랜차이즈 전체	본사 + 매장 묶음
본사	Control Plane	본사	클러스터 두뇌
	API Server	본사 접수창구	모든 지시·보고의 유일한 창구
	etcd	장부 금고	클러스터 상태 전체 기록
	Scheduler	인력 배치 담당	새 Pod를 어느 노드에 둘지 결정
	Controller Manager	운영 감독관	지침과 현장이 다르면 보정
매장	Node	매장	Pod가 실제로 도는 서버
	kubelet	매장 점장	본사 지시대로 컨테이너 실행·보고
	kube-proxy	매장 전화 교환기	Service 주소 → Pod 연결
	Container Runtime	주방 설비	컨테이너 실행 엔진
일하는 단위	Pod	조리 스테이션(직원 1조)	컨테이너 묶음, 최소 배포 단위
	ReplicaSet	정원 규칙	Pod 개수 유지
	Deployment	메뉴 운영 지침서	버전 교체·롤백
연결	Service	대표 전화번호	바뀌지 않는 접속 주소
	Ingress / Controller	푸드코트 안내판 / 안내 직원	도메인·경로별 분기
저장	PV / PVC / StorageClass	창고 / 사용 신청서 / 창고 카탈로그	영구 데이터 저장
운영 규칙	Namespace / Label	브랜드 구역 / 명찰	구역 분리 / 대상 지정
접근	kubeconfig / kubectl	본사 출입증 / 업무 단말	API Server 접속 정보 / 명령 도구

3. Control Plane - 본사 4개 부서



구성 요소	비유	하는 일	멈추면
kube-apiserver	접수창구	모든 요청의 단일 진입점. 인증 → 인가 → 검증 → etcd 저장	kubectl·컨트롤러 모두 멈춤
etcd	장부 금고	클러스터 모든 오브젝트 상태를 Key-Value로 저장	손실 시 "무엇이 어디서 돌아야 하는지" 기록 소멸
kube-scheduler	인력 배치 담당	노드 미배정 Pod의 배치 노드 결정	새 Pod가 Pending에 머묾
kube-controller-manager	운영 감독관	원하는 상태와 현재 상태를 비교해 보정	자동 복구·개수 유지 중단
cloud-controller-manager	건물 관리사무소 연락 담당	클라우드 LB·노드 정보 연동	type LoadBalancer Service 생성 불가

Scheduler가 보는 조건

- 자원: Pod가 요청한 CPU·메모리(requests)를 받아 줄 여유가 있는 노드
- 지정: nodeSelector·nodeAffinity("GPU 노드에만"), podAntiAffinity("같은 앱은 다른 노드에 흩어서")
- 제한: taint/toleration("특수 매장에는 허가받은 직원만")

> **구조상 알아 둘 점**
>
> - 구성 요소끼리 직접 통신하지 않고 모두 API Server를 거침 (Scheduler가 etcd를 직접 읽지 않음)
> - Control Plane이 잠시 멈춰도 이미 떠 있는 Pod는 계속 서비스. 본사 전화가 끊겨도 매장은 영업 중인 상태와 같음. 다만 새 배포·자동 복구는 불가
> - 운영 환경의 etcd·API Server는 3중화 + etcd 백업이 기본 → 관리형 서비스(13장)에서는 이 부분을 클라우드가 맡음

4. Node - 매장과 점장

구성 요소	비유	하는 일
kubelet	매장 점장	API Server에서 내 노드에 배정된 Pod를 받아 실행, 헬스체크(Probe) 수행, 상태 보고
kube-proxy	매장 전화 교환기	Service IP로 온 요청을 실제 Pod IP로 넘기는 규칙(iptables/IPVS) 관리
Container Runtime	주방 설비	이미지를 받아 컨테이너 실행 (containerd 등)
CNI 플러그인	매장 사이 배달 도로	노드가 달라도 Pod끼리 통신하게 하는 네트워크 (SCP는 Calico)

노드 장애 시 흐름

```
노드 장애 → kubelet 보고 끊김 → Node 상태 NotReady
 → 일정 시간 후 해당 노드의 Pod를 다른 노드에 재생성 (Deployment 등이 관리하는 Pod만)
```

- 컨트롤러 없이 만든 단독 Pod는 다시 만들어 주는 주체가 없음 → 항상 Deployment 등으로 배포
- 노드 = 결국 VM 또는 물리 서버. SCP에서는 노드가 Virtual Server 목록에 그대로 조회됨

노드 자원을 Pod가 전부 쓰지 못하는 이유 - kube-reserved

- kubelet·컨테이너 런타임 같은 시스템 데몬 몫을 노드 자원에서 먼저 떼어 둠
- SCP 매뉴얼 예약 공식(구간 누적): CPU 첫 1코어 6%, 다음 1코어 1%, 다음 2코어 0.5%, 초과분 0.25% / 메모리 첫 4GB 25%, 다음 4GB 20%, 다음 8GB 10%, 다음 112GB 6%, 초과분 2%

노드	CPU 예약	메모리 예약	출처
vCPU 16 / 32GB	약 0.11코어	약 3.56GB	SCP 매뉴얼 예시
s1v2m4 (vCPU 2 / 4GB, 실습 노드)	약 0.07코어	약 1GB	공식에 직접 대입한 값

- 작은 노드일수록 예약 비율이 큼 → 4GB 노드는 메모리 1/4을 시스템이 사용. 노드 스펙을 정할 때 고려

5. 선언형 운영과 Reconcile Loop - 지침서와 순찰

비유: 본사는 "3번 튀김기 켜" 같은 개별 명령(명령형)이 아니라 "스테이션 3개 상시 운영" 같은 지침서(선언형)를 내려보냄. 운영 감독관은 매장을 계속 순찰하며 지침과 다르면 맞춤

구분	명령형	선언형 (쿠버네티스 방식)
지시 내용	"컨테이너 1개 더 띄워"	"항상 3개 떠 있어야 함"
장애 후	사람이 다시 명령	시스템이 차이를 보고 자동 보정
기록	명령 이력만 남음	YAML 자체가 현재 구성의 정답지

kubectl apply 한 번에 일어나는 일

```
1. kubectl apply -f deploy.yaml
2. API Server : 인증 · 인가 · 검증 → etcd에 Deployment 저장
3. Controller Manager : Deployment → ReplicaSet 생성 → Pod 3개 생성 요청
4. Scheduler : 노드 미배정 Pod 3개의 배치 노드 결정
5. kubelet : 자기 노드에 배정된 Pod의 컨테이너 실행, 상태 보고
6. 이후 계속 : 현재 상태 ≠ 원하는 상태이면 3~5 반복 (Reconcile Loop)
```

Reconcile Loop 4단계: 현재 상태 확인 → 원하는 상태 확인 → 차이 계산 → 보정 → 처음으로

> **선언형의 뒷면 - 정답지 밖에서 바꾼 값은 되돌아감**
>
> - 컨트롤러는 YAML(원하는 상태)만 기준으로 삼음. 화면이나 명령으로 결과물만 직접 고치면 다음 순찰 때 원래대로 복구
> - 예: Deployment가 관리하는 Pod를 지워도 곧 새로 생김 / 클라우드 LB를 콘솔에서 직접 고쳐도 Service 기준으로 원복
> - 원칙: 고칠 때는 결과물이 아니라 정답지(YAML·annotation)를 고침 (SCP 사례는 18장)

6. 워크로드 오브젝트 - Pod는 누가 관리하나

```
Deployment (메뉴 운영 지침서: 버전 · 개수 · 교체 방식)
  └ ReplicaSet (정원 규칙: "스테이션 3개 유지")
       └ Pod  Pod  Pod (조리 스테이션)
            └ 컨테이너 (+ 사이드카 컨테이너)
```

Pod (조리 스테이션)

- 최소 배포 단위. 컨테이너 1개 이상을 묶음
- 같은 Pod의 컨테이너는 같은 노드에서 IP(localhost)·볼륨을 공유 = 같은 스테이션은 도마·냉장고 공유
- 사이드카 = 같은 스테이션의 보조 직원 (로그 수집, 프록시 등)
- Pod IP는 재생성될 때마다 바뀜 → 고정 주소는 Service가 담당(8장)

ReplicaSet / Deployment

- ReplicaSet: 지정한 개수만큼 Pod 유지. 하나가 죽으면 새로 생성
- Deployment: ReplicaSet을 버전별로 만들고 교체하며 롤링 업데이트·롤백 담당
- 실무에서는 Pod·ReplicaSet을 직접 만들지 않고 Deployment만 작성

그 밖의 워크로드

오브젝트	비유	쓰는 곳
StatefulSet	지정석 직원 (이름표·전용 사물함 고정)	DB, 메시지 큐처럼 순서·고유 이름·전용 볼륨이 필요한 앱
DaemonSet	매장마다 1명씩 두는 위생 점검원	노드마다 하나씩 필요한 로그·모니터링 에이전트, CNI
Job	한 번 하고 끝나는 대청소	일회성 배치, 데이터 이관
CronJob	매일 밤 마감 정산	주기적 배치, 백업

7. 무중단 배포 · 자동 복구 · 헬스체크

롤링 업데이트 (Deployment 기본 방식)

```
[기존]      v1  v1  v1
[교체 중]   v1  v1  v2   ← v2 하나 투입, 준비되면 v1 하나 제거
              v1  v2  v2
[완료]      v2  v2  v2   ← 서비스 중단 없이 교체
```

- maxSurge: 교체 중 정원보다 더 띄울 수 있는 수 / maxUnavailable: 교체 중 빠져도 되는 수
- 롤백: `kubectl rollout undo deployment/<이름>` → 이전 ReplicaSet으로 복귀. 이전 버전 ReplicaSet이 남아 있어 빠르게 전환

헬스체크 3종 (Probe) - 직원 컨디션 체크

Probe	비유	실패 시
livenessProbe	쓰러졌는지 확인	컨테이너 재시작
readinessProbe	손님 받을 준비가 됐는지 확인	재시작 없이 Service 대상에서만 제외
startupProbe	출근 직후 준비 시간 보장	기동 완료 전까지 liveness 검사 유예 (기동이 느린 WAS)

자동 복구 단계

장애 범위	복구 주체	동작
컨테이너 종료	kubelet	같은 Pod 안에서 재시작. 반복 실패 시 대기 시간이 늘어나는 CrashLoopBackOff
Pod 삭제	ReplicaSet	개수가 모자라면 새 Pod 생성
노드 장애	Controller Manager + Scheduler	다른 노드에 Pod 재생성
노드 자체 교체	클라우드 (SCP 노드 자동 복구)	장애 노드를 새 VM으로 교체 (15장)

> **readinessProbe가 없으면**
>
> - 롤링 업데이트 중 아직 기동 중인 Pod에도 트래픽이 들어가 502·503 발생 → "무중단 배포"가 실제로는 순단
> - liveness를 너무 빡빡하게 잡으면 일시적 부하에도 재시작이 반복되어 장애가 커짐

8. Service - 대표 전화번호

문제: Pod는 재생성될 때마다 IP가 바뀜 → IP로 연결하면 끊김

해결: Service = 바뀌지 않는 가상 IP + DNS 이름(`<서비스>.<네임스페이스>.svc.cluster.local`)

- 대상 Pod는 IP가 아니라 Label Selector로 지정 → Pod가 바뀌어도 대상 목록(Endpoints)이 자동 갱신
- 비유: 직원(Pod)이 바뀌어도 매장 대표번호는 그대로, 교환기(kube-proxy)가 지금 일하는 직원에게 연결

Service 타입

타입	비유	접근 범위	비고
ClusterIP	내선번호	클러스터 내부	기본값. 내부 서비스 간 통신
NodePort	매장 뒷문 번호	모든 노드의 같은 포트 (30000~32767)	외부 LB를 수동으로 붙일 때의 접점
LoadBalancer	대표번호 + 콜센터	클라우드 LB를 통한 외부 접근	클라우드가 LB를 자동 생성
ExternalName	다른 회사 번호로 착신 전환	외부 DNS 이름으로 연결	외부 DB 등

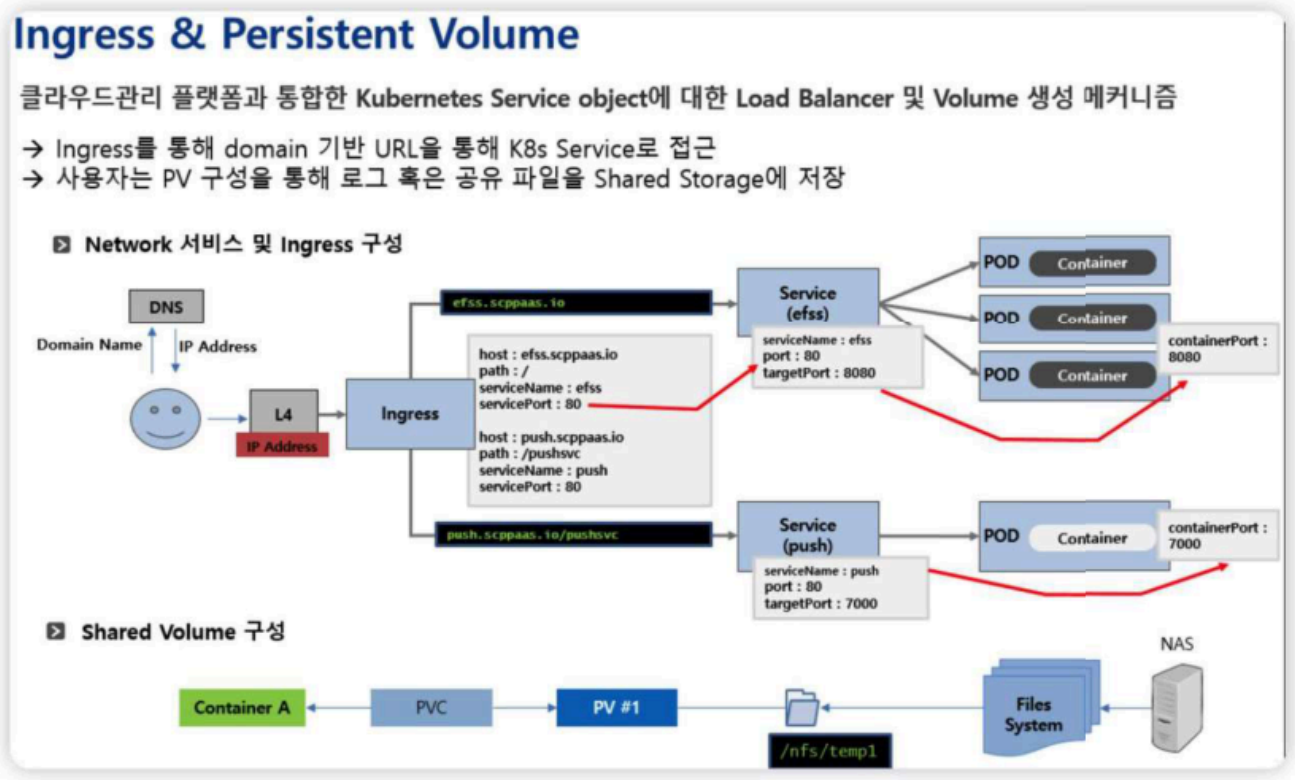
- 포함 관계: ClusterIP ⊂ NodePort ⊂ LoadBalancer. LoadBalancer는 NodePort를 만들고 그 앞에 클라우드 LB를 붙이는 구조

포트 용어 (HOL2 교안 그림 기준)

용어	위치	예
port (servicePort)	Service가 받는 포트	80
targetPort	Service가 넘겨주는 Pod 포트	8080
containerPort	컨테이너 앱이 실제로 여는 포트	8080
nodePort	노드에서 외부로 열리는 포트	32615 (실습값)

- targetPort와 containerPort가 다르면 Service는 정상이어도 연결 실패 → 장애 시 먼저 확인할 짝

9. Ingress - 푸드코트 안내판과 안내 직원



비유

- Ingress(리소스) = 푸드코트 안내판. "efss.scppaas.io로 온 손님 → efss 매장"처럼 도메인·경로별 규칙만 적힌 문서
- Ingress Controller = 안내판을 읽고 손님을 실제로 데려가는 안내 직원 (nginx 등 프록시 Pod)
- 안내판만 있고 직원이 없으면 아무 일도 안 일어남 / 직원만 있고 안내판이 없으면 "갈 곳 없음" = 404

그림의 라우팅 규칙

Host / Path	Service	servicePort → targetPort
efss.scppaas.io /	efss	80 → 8080
push.scppaas.io /pushsvc	push	80 → 7000

Service(LoadBalancer)만 쓸 때와 비교

항목	Service 타입 LoadBalancer	Ingress
LB 수	서비스마다 1개	LB 1개로 여러 서비스 처리
분기 기준	IP·포트 (L4)	도메인·URL 경로 (L7)
TLS 인증서	서비스별 관리	Ingress에서 한 번에 종료·관리
비용	서비스 수에 비례	거의 고정

- IngressClass: 클러스터에 컨트롤러가 여럿일 때 "어느 안내 직원이 이 안내판을 맡을지" 지정 (실습값: nginx)
- 외부 흐름: 클라이언트 → 클라우드 LB → Ingress Controller → (규칙) → Service → Pod

> **Ingress는 L7, Service는 L4**
> - Service는 IP·포트만 보고 넘김 / Ingress Controller는 HTTP Host 헤더·URL 경로까지 보고 분기
> - 그래서 Ingress는 도메인 이름으로 접속해야 규칙에 맞음. IP로 직접 접속하면 Host가 맞지 않아 404

10. 스토리지 - 창고, 신청서, 창고 카탈로그

문제: 컨테이너 안에 저장한 파일은 Pod가 사라지면 같이 사라짐

오브젝트	비유	담당	역할
PV (PersistentVolume)	실제 창고	관리자 또는 자동 생성	실제 저장 공간(디스크, NAS)
PVC (PersistentVolumeClaim)	창고 사용 신청서	개발자	"10GB, 여러 명이 같이 쓰는 창고" 요청
StorageClass	창고 규격 카탈로그	관리자	신청서가 들어오면 해당 규격 창고를 자동 설치 (동적 프로비저닝)

```
Pod → PVC(신청서) → [StorageClass가 자동 생성] → PV(창고) → 실제 스토리지(Block / NAS)
```

AccessMode

모드	의미	비유	대표 스토리지
ReadWriteOnce (RWO)	한 노드에서만 읽기·쓰기	1인용 개인 사물함	블록 스토리지
ReadOnlyMany (ROX)	여러 노드에서 읽기만	공용 열람실	NAS
ReadWriteMany (RWX)	여러 노드에서 읽기·쓰기	공용 창고	NAS(NFS)

- 회수 정책(reclaimPolicy): Delete = 신청서 반납 시 창고도 철거 / Retain = 창고 보존 → 운영 데이터는 Retain 여부 확인

SCP 매핑 (SCP 매뉴얼·HOL2 화면 기준)

SCP 상품	쿠버네티스 연결	AccessMode	비고
Block Storage	Block Storage CSI 스토리지 클래스 `bs-ssd` (기본), `bs-ssd-xfs`, `bs-ssdprovisioned-sc` 등	RWO	기본 파일시스템 ext4, 클래스별 볼륨 확장 지원
File Storage	NFS 기반 스토리지 클래스 (HOL2 화면: `nfs-subdir-external-sc`)	RWX	클러스터 생성 시 연결. 실습 앱의 로그 PVC가 여기에 생성
Object Storage	PV 방식이 아니라 앱에서 S3 호환 API로 접근	-	노드풀당 연결 3개 제한 (매뉴얼)

11. 운영 규칙 오브젝트 - 구역 · 명찰 · 설정표 · 출입 권한

오브젝트	비유	역할
Namespace	브랜드별 구역	리소스 이름·권한·쿼터를 나누는 논리 구역
Label / Selector	직원 명찰 / 명찰로 찾기	`app=tomcat` 같은 표시로 Service·Deployment가 대상 Pod를 고름
Annotation	업무 메모지	동작 설정을 붙이는 부가 정보. SCP LB 옵션도 annotation으로 지정(18장)
ConfigMap	매장 운영 설정표	환경별 설정값을 이미지 밖으로 분리
Secret	봉인 봉투	비밀번호·인증서. base64 인코딩일 뿐 암호화가 아님 → 접근 권한 통제 필수
ResourceQuota / LimitRange	구역별 예산 한도	네임스페이스가 쓸 수 있는 CPU·메모리 상한
ServiceAccount	Pod의 사원증	Pod가 API Server를 호출할 때의 신원
Role / ClusterRole + Binding	출입 권한표	RBAC. 누가(주체) 어떤 리소스에 무엇을(get·create·delete) 할 수 있는지
NetworkPolicy	구역 간 출입 통제	Pod 간 통신 허용·차단. CNI가 지원해야 동작 (Calico 지원)

기본 Namespace (HOL2 `kubectl get ns` 결과)

Namespace	용도
default	지정 없이 만든 리소스
kube-system	시스템 구성 요소 (DNS, CNI 등)
kube-public	누구나 읽을 수 있는 공개 정보
kube-node-lease	노드 생존 신호(heartbeat)
ingress, tomcat	실습에서 Kubernetes Apps가 만든 구역

기본 상태는 "모두 통함"

- NetworkPolicy가 없으면 모든 Pod가 서로 통신 가능 → 웹 Pod 하나가 침해되면 DB Pod까지 바로 접근
- 권장 순서: 네임스페이스별 기본 차단 정책 → 필요한 통신만 허용 추가
- SCP 매뉴얼: Kubernetes Engine은 쿠버네티스 RBAC와 SCP IAM을 연계해 사용자 권한을 관리

12. 자원과 확장 - 정원 관리와 매장 증설

requests / limits

항목	비유	의미	넘으면
requests	예약석	스케줄러가 배치 판단에 쓰는 최소 보장량	받아 줄 노드가 없으면 Pending
limits	최대 허용량	컨테이너가 쓸 수 있는 상한	CPU는 속도 제한(throttling), 메모리는 강제 종료(OOMKilled)

- 실습값: Tomcat vCPU 0.2 / Memory 0.1, Ingress Controller vCPU 0.5 / Memory 1 → 노드(s1v2m4 2대) 여유 안에서만 배치 가능

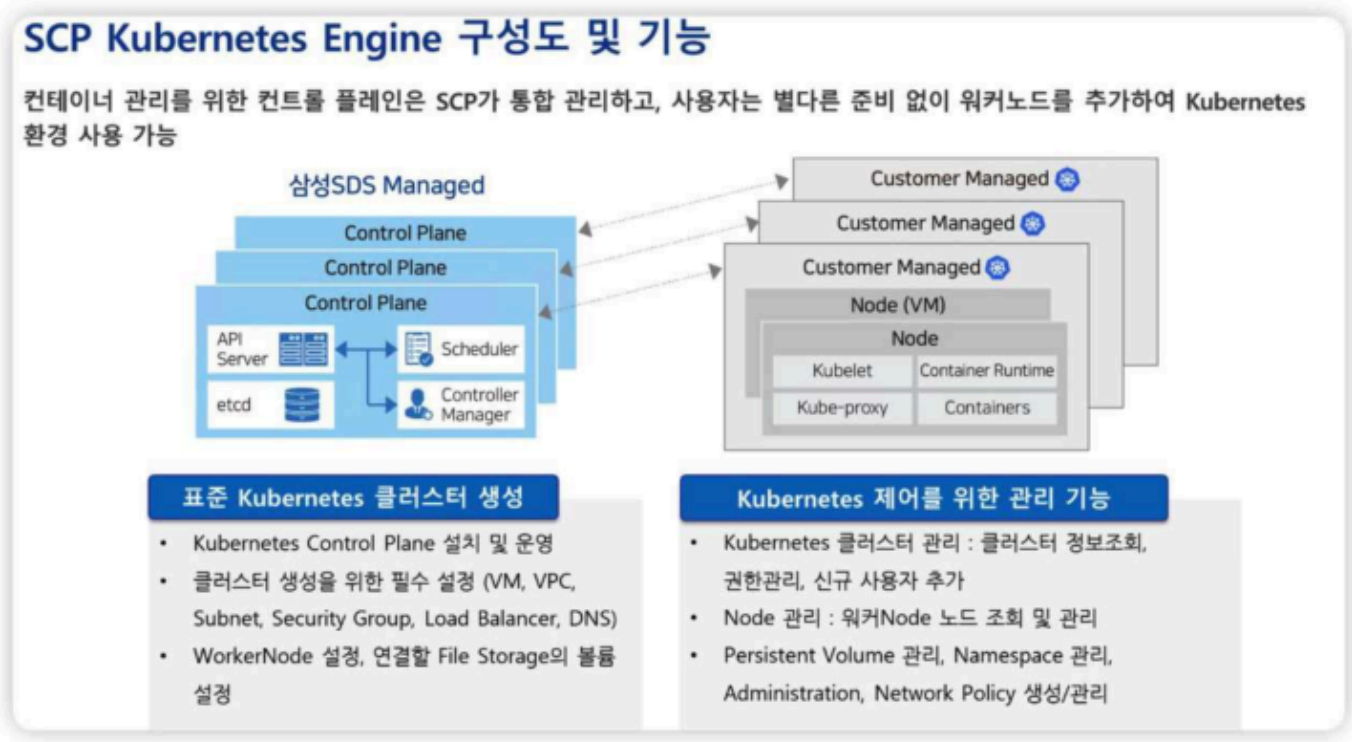
확장 3단계

단계	비유	쿠버네티스	SCP
Pod 수	스테이션 증설	HPA (CPU 사용률 등으로 Pod 수 조정)	Kubernetes Apps 배포 시 Auto Scaling 옵션
Pod 크기	스테이션 장비 교체	VPA (requests 자동 조정)	-
노드 수	매장 증설	Cluster Autoscaler	노드 풀 자동 확장/축소

```
트래픽 증가 → HPA가 Pod 증설 → 노드 자원 부족 → 새 Pod Pending
 → 노드 풀 자동 확장 → 새 노드에 Pending Pod 배치
```

- Pod 증설은 수 초, 노드 증설은 VM 생성이라 수 분 → 노드에 어느 정도 여유를 남겨 두는 설계 필요
- 축소 시 노드에서 Pod를 옮기는 과정이 생김 → Pod가 2개 이상이고 readiness가 있어야 순단 없음

13. SCP Kubernetes Engine - 본사 위탁 운영



비유: 본사(Control Plane) 운영은 SCP가 대행하고, 고객은 매장(노드)과 메뉴(워크로드)만 운영하는 위탁 운영형 프랜차이즈

영역	구성	관리 주체
Control Plane	API Server, etcd, Scheduler, Controller Manager (이중화 구성)	삼성SDS (SDS Managed)
Node (VM)	kubelet, kube-proxy, Container Runtime	고객 (Customer Managed)
워크로드	Pod, Service, Ingress, PVC 등	고객

SCP가 맡는 일 (교안)

- 표준 클러스터 생성: Control Plane 설치·운영, 클러스터 필수 설정(VM, VPC, Subnet, Security Group, Load Balancer, DNS), Worker Node 설정과 File Storage 볼륨 연결
- 콘솔 관리 기능: 클러스터 정보 조회·권한 관리·사용자 추가, 노드 조회·관리, PV·Namespace 관리, Network Policy 생성·관리
- 매뉴얼 특장점: 표준 쿠버네티스와 호환 → 다른 표준 환경의 애플리케이션을 코드 수정 없이 사용

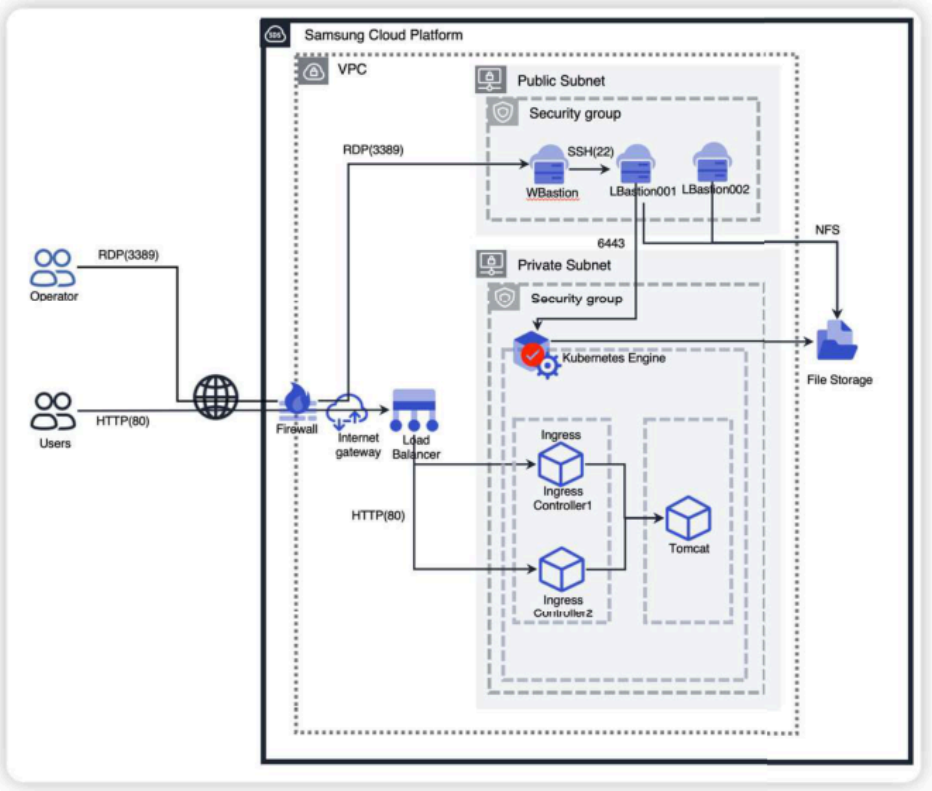
선행 상품: VPC, Security Group, Load Balancer, File Storage → 쿠버네티스도 결국 VPC 위의 VM·LB·NAS 조합

SCP 컨테이너 관련 상품

상품	비유	역할
Kubernetes Engine (SKE)	위탁 운영 본사 + 매장	관리형 쿠버네티스 클러스터
Kubernetes Apps	본사 추천 메뉴 카탈로그	검증된 앱(Ingress Controller, Tomcat 등)을 콘솔에서 바로 배포
Container Registry	밀키트 물류창고	컨테이너 이미지 저장·버전 관리
DevOps Service	신메뉴 출시 라인	빌드·배포 파이프라인 (7월 자료)

> **관리형이어도 고객 몫으로 남는 것**
>
> - 노드는 고객 VM → 노드 수·스펙·OS·Security Group·비용 관리는 Virtual Server와 동일
> - 클러스터 버전 업그레이드 시점 결정, 워크로드 설계(Probe, requests, 이중화)도 고객 책임
> - `kubectl get node` 에 Worker Node만 보이고 Control Plane 노드는 보이지 않음 (16장)

14. 실습 구성도 - 무엇을 어디에 왜 두었나



구성 요소	배치	설계 의도
Firewall + Internet Gateway	VPC 경계	인터넷 출입구를 한 곳으로
Load Balancer	VPC 안	사용자 HTTP(80)는 LB로만 → 노드 직접 노출 없음
WBastion (Windows)	Public Subnet	운영자 RDP(3389) 진입점
LBastion001·002 (Linux)	Public Subnet	kubectl 실행 서버. API Server(6443) 접근 허용 대상
Kubernetes Engine 노드	Private Subnet	노드에 공인 IP 없음
Ingress Controller x2	노드 위 Pod	LB 뒤 L7 분기, 2개로 이중화
Tomcat	노드 위 Pod	실제 애플리케이션
File Storage	VPC	NFS로 노드에 연결, PVC로 로그 저장

두 경로가 분리된 구조

```
사용자 경로 : Users → Firewall/IGW → LB(80) → Ingress Controller → Tomcat
운영자 경로 : Operator → RDP → WBastion → SSH(22) → LBastion → 6443 → API Server
```

- 9월 자료(IaaS 구성)와 같은 원칙: 최소 노출(인터넷에서 닿는 곳은 LB·Bastion), 사용자·운영자 경로 분리
- API Server도 "관리 포트"로 취급 → 22·3389와 마찬가지로 Bastion에서만 접근
- 노드가 Private에 있어도 이미지 다운로드 등 외부 통신이 필요하면 NAT Gateway·Firewall Outbound 규칙 필요 (9월 4장)

HOL1 튜토리얼과 차이: HOL1은 Bastion 1대(Linux, Public Subnet, NAT IP)에서 MobaXterm으로 SSH 접속해 같은 작업 수행

15. 클러스터 · 노드 풀 생성 - 처음에 정하는 것

클러스터 설정 (교안 입력값)

항목	HOL1 (2024.10)	HOL2	의미
Kubernetes 버전	v1.26.8	v1.24.8	kubectl 버전 기준이 됨
프라이빗 엔드포인트	Bastionxx 등록	LBastion001·002 허용	API Server 접근 허용 VM 목록
퍼블릭 엔드포인트	사용 안 함	사용 안 함	인터넷에서 API Server 접근
제어 영역 로깅	사용	사용 안 함	API 호출 이력 기록
VPC / 서브넷	VPCxx / Private	VPCxx / PRISUBxx	노드가 생성될 위치
Security Group	WebSGxx	k8sSGxx	노드에 적용
Load Balancer	LBxx	LBxx	type LoadBalancer·Ingress 연결용
File Storage	fsxx	fsxx	PV로 쓸 NAS 볼륨

- 상태 흐름: 클러스터 Creating → Running / 노드 풀 Scaling Up → Running. 멈추면 상세의 노드 정보에서 Failed 확인

노드 풀 설정 (교안 입력값): OS Ubuntu, 노드 수 2, STANDARD s1v2m4, Block Storage SSD 100GB, 자동 확장/축소 미사용, 노드 자동 복구 미사용 → 노드 생성 결과·접속 계정은 문자·메일로 안내

노드 풀 규칙 (SCP 매뉴얼)

- 한 노드 풀의 노드는 서버 타입·크기·OS 이미지가 같음 → 성격이 다른 워크로드(GPU, 고메모리)는 노드 풀을 나눔
- 라벨·taint 설정은 이후 생성 노드에만 적용, 기존 노드에는 소급되지 않음
- 노드 수를 바꾸면 노드가 추가·삭제되며 컨테이너가 다른 노드로 이동 → 서비스 중단 가능 경고
- 노드 풀당 File Storage 3개, Object Storage 3개 연결 제한

버전 수명 (SCP 매뉴얼, 2026-11 기준)

구분	내용
업스트림 주기	연 3회 마이너 출시, 각 약 14개월 지원
SKE 제공	Stable 패치 검증 후 제공 → 업스트림보다 출시가 늦음
기술지원 종료(EoTS) 후	신규 클러스터 생성 불가. 기존 클러스터 업그레이드·노드 풀 생성은 가능
현재	v1.31 EoTS 2026-10-28 경과 → 신규 생성은 v1.32~v1.36. 교안 버전(v1.24·v1.26)은 생성 불가

처음에 잘 정해야 하는 값

- VPC·서브넷: 노드와 LB가 올라간 뒤에는 사실상 재구축. 노드 수 증가에 맞춰 서브넷 IP 여유 확보
- 노드 풀 스펙: 기존 노드 풀의 서버 타입 변경 절차가 매뉴얼에 없음 → 새 노드 풀 생성 후 이동이 현실적인 방법
- 버전: EoTS가 다가오면 업그레이드 계획 필요 (콘솔 업그레이드, 별도 비용 없음, 사전 호환성 테스트 권장)

16. API Server 접근 - 본사 출입증(kubeconfig)

접근 경로: 내 PC → (RDP) WBastion → (SSH) LBastion → kubectl → (6443) 프라이빗 엔드포인트 → API Server

단계별 개념

단계	내용	비유
kubectl 설치	클러스터 버전과 맞춰 설치. 지원 범위 ±1 마이너 버전	본사 업무 단말 준비
접속 실패 확인	kubeconfig 없이 `kubectl version` → `localhost:8080 refused`	단말만 켜고 출입증이 없는 상태
엔드포인트 설정	콘솔 클러스터 상세에서 프라이빗 엔드포인트 사용 + Bastion 등록	출입 허용 명단 등록
kubeconfig 저장	콘솔의 kubeconfig 내용을 `~/.kube/config` 에 저장	출입증 발급
확인	`kubectl version` 에 Server 버전 표시, `kubectl get node`	본사 연결 확인

kubeconfig 구조

항목	내용	비유
clusters	API Server 주소(`https://<클러스터>...:6443`)와 CA 인증서	본사 주소
users	클라이언트 인증서·키 또는 토큰	출입증
contexts	어느 클러스터에 어느 사용자로 접속할지	출입 조합
current-context	현재 사용 중인 조합	지금 들고 있는 출입증

HOL2 확인 결과에서 읽을 수 있는 것

- `kubectl get node` → 노드 2대, ROLES `<none>`, 버전 `v1.22.5-ske.p3` → Worker Node만 보임 = Control Plane은 SCP 관리 영역
- 화면의 Client v1.21.8 / Server v1.22.5 → 1 마이너 차이라 지원 범위 안
- 프라이빗 vs 퍼블릭 엔드포인트: VPC 안(허용 VM)에서만 접근 / 인터넷에서 접근(허용 IP 지정). 실습은 프라이빗만 사용

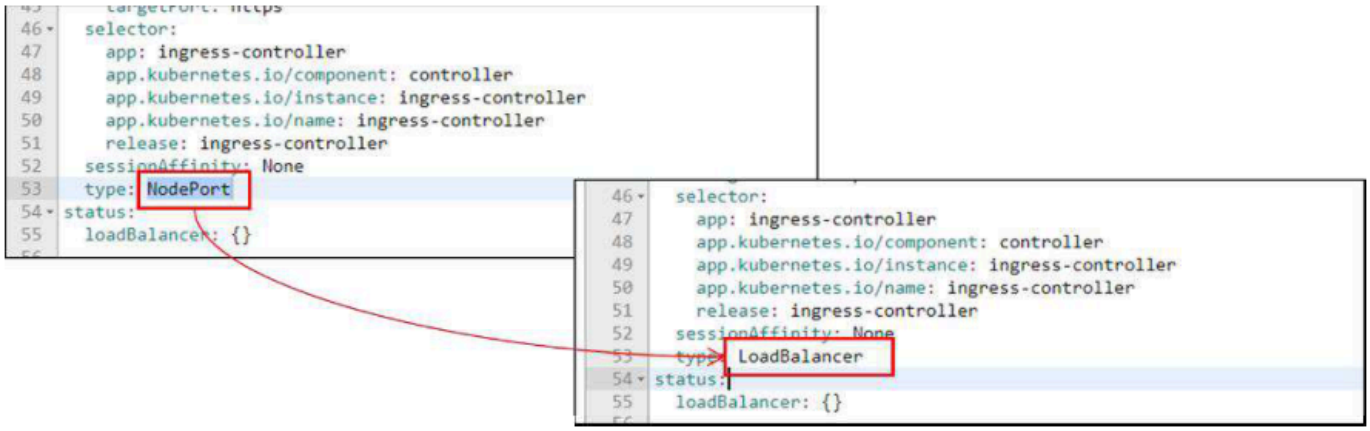
실무 확인 사항 (외부 지식 보강, 매뉴얼 미기재)

- 관리자용 kubeconfig는 최초 1회만 다운로드됨 → 받는 즉시 안전한 서버에 보관
- 퍼블릭 엔드포인트 설정은 반영까지 수 분(5분 이상) 걸리는 비동기 작업 → 저장 직후 접속이 안 돼도 재시도 전에 대기

> **kubeconfig = 클러스터 권한 그 자체**
>
> - 파일 하나로 클러스터 전체를 조작할 수 있음 → 메일·메신저 평문 전달 금지, Bastion 등 통제된 서버에만 보관
> - 사용자별로 권한을 나눌 때는 관리자 kubeconfig 공유가 아니라 SCP IAM 연계 + RBAC로 사용자 권한을 따로 부여

17. 외부 노출 - 수동 연결(HOL1) vs 자동 연결(HOL2)

트래픽 경로: 사용자 → LB 서비스(80) → 노드:NodePort → Ingress Controller Pod → (Ingress 규칙) → Service → Tomcat Pod





항목	HOL1 튜토리얼 - 수동 연결	HOL2 - 자동 연결
Ingress Controller Service 타입	NodePort 유지	YAML에서 NodePort → LoadBalancer 변경
LB 서버그룹	직접 생성: 노드 VM 2대 + NodePort(32615), 헬스체크도 NodePort	자동 생성
LB 프로파일·서비스	직접 생성: 애플리케이션 프로파일(세션 유지 300초), 서비스 80 → 32615, NAT IP 사용	자동 생성: 80 → 31985, 443 → 30323 (L4/TCP)
External-IP	`<none>` (LB의 NAT IP로 접속)	Service에 자동 표시
누가 관리하나	사람 (쿠버네티스 밖의 LB)	쿠버네티스 (cloud-controller-manager)
노드 증감 시	서버그룹을 사람이 갱신	서버그룹 멤버 자동 동기화 (실무 관찰)
콘솔에서 직접 수정	가능	원복 위험 (18장)

결과 확인과 오류 해석

- Ingress Controller만 배포하고 앱이 없을 때 External-IP 접속 → nginx 404 = LB·NodePort·Controller까지는 정상, 연결할 규칙만 없음
- Tomcat 배포 시 Service 타입 Ingress, Class nginx, 도메인 지정 → 그 도메인으로 접속해야 Host 규칙에 맞음
- 도메인 연결: HOL1은 PC hosts 파일에 External-IP·도메인 등록 / HOL2는 운영자가 DNS A 레코드 등록(`tomcat-ingressxx.scpspecialist.com` → Ingress External-IP)
- 사내망에서는 방화벽 예외 허용이 있어야 접속 확인 가능
- 오류 위치로 도달 계층 판단: 응답 없음(LB·Firewall·SG) → 502(Controller는 받았지만 Pod 응답 없음) → 404(Controller 도달, 규칙 불일치)

18. type LoadBalancer 설정과 자동 생성 LB 운영

동작 (SCP 매뉴얼)

- `type: LoadBalancer` Service 적용 → 대응하는 LB 서비스·LB 서버그룹 자동 생성 (수 분 소요), 기본 위치는 클러스터 Subnet
- 설정은 콘솔이 아니라 Service의 annotation으로 지정

주요 annotation (접두어 `service.beta.kubernetes.io/` 생략)

annotation	기본값	의미	생성 후 변경
scp-load-balancer-layer-type	L4	L4(TCP·UDP·TLS) / L7(HTTP·HTTPS). 한 Service에 혼용 불가	불가 (재생성)
scp-load-balancer-subnet-id	클러스터 Subnet	LB가 속할 Subnet	불가
scp-load-balancer-service-ip	-	LB 서비스 IP 지정	불가
scp-load-balancer-zones	-	AZ 최대 2개 지정	불가
scp-load-balancer-public-ip-enabled	false	Public NAT IP 사용 (Public IP는 AZ1에 생성)	-
scp-load-balancer-client-cert-id	-	HTTPS 인증서 ID. L7 HTTPS면 필수	-
scp-load-balancer-source-ranges-firewall-rules	false	허용 출발지 → LB 방화벽 규칙 자동 추가	-
scp-load-balancer-snat-healthcheck-firewall-rules	false	LB SNAT·헬스체크 IP → 멤버 규칙 자동 추가	-
scp-load-balancer-security-group-id / -name	-	지정 SG에 규칙 자동 추가	-

운영 시 주의 (매뉴얼 경고 + 실무)

- 매뉴얼: 자동 생성된 LB 서비스·서버그룹을 임의로 변경하지 말 것 → 변경이 원복되거나 예기치 않은 동작
- 이유는 5장의 Reconcile: 컨트롤러의 정답지는 Service manifest. 콘솔 수정은 정답지 밖의 변경이라 다음 동기화(서비스 수정, 노드 증감, 컨트롤러 재시작)에 원복
- 예: 인증서를 콘솔에서만 교체 → annotation의 cert-id는 옛 값 → 며칠 뒤 옛 인증서로 되돌아가 장애
- 방화벽 자동 추가는 포트 수만큼 규칙이 생겨 많아질 수 있음 → NodePort 대역(30000~32767) 기준 수동 규칙도 선택지
- SG annotation으로 추가된 규칙은 annotation을 지우거나 바꿔도 자동 삭제되지 않음 → 수동 정리 필요

> **LB를 고치기 전에 먼저 물을 것: "이 LB를 쿠버네티스가 만들었나"**
>
> - `kubectl get svc -A | grep LoadBalancer`의 EXTERNAL-IP가 해당 LB IP와 같으면 쿠버네티스 관리 대상
> - 관리 대상이면 콘솔이 아니라 Service annotation을 수정 후 kubectl apply
> - HOL1처럼 사람이 만들어 NodePort로 붙인 LB는 쿠버네티스가 관리하지 않으므로 원복 위험 없음

19. Kubernetes Apps · 스토리지 연계 · 삭제 순서

Kubernetes Apps - 본사 추천 메뉴 카탈로그

- 클러스터·Namespace를 고르고 앱을 선택해 콘솔에서 바로 배포 (SCP 2.0 기준 30개, SCP 3.0 기준 39개)
- 실습 배포값

앱	Namespace	Pod 수	자원 (vCPU / Memory)	Service	PVC
NGINX Ingress Controller Community	ingress	2 (HOL2)	0.5 / 1	NodePort로 생성	사용
Tomcat Community	ingress·tomcat	1	0.2 / 0.1	Ingress (Domain, Class nginx)	logs (기본값)

- 앱 상세 화면에서 Service·Ingress·External-IP·포트 매핑 확인 → kubectl 없이도 상태 파악 가능
- Pod 수는 노드 풀 자원 여유 안에서만 늘릴 수 있음 (실습은 자원 제약으로 1개)

스토리지 연계

- 클러스터 생성 시 File Storage 볼륨 연결 → 앱의 PVC가 NFS 스토리지 클래스로 생성되어 로그 저장
- Block Storage가 필요한 앱(DB 등 RWO)은 `bs-ssd` 계열 스토리지 클래스로 PVC 생성

삭제 순서 (자원 의존성)

```
1. Kubernetes Apps 삭제 (Tomcat, Ingress Controller) → 실제 Pod 삭제
2. 노드 풀 삭제 → 총 노드 수 0 (Deleting, 수 분 소요) → Virtual Server 목록에서도 사라짐
3. 클러스터 상품 해지 = Control Plane 삭제 (Terminating → 목록에서 제거)
4. File Storage는 별도 삭제 (클러스터 삭제 시 적용 서버 연결만 해제되고 볼륨은 남음)
5. 이후 LB, Security Group, Bastion, Subnet, VPC 순으로 정리
```

- 노드 풀이 남아 있으면 클러스터 해지 불가 → 매장을 먼저 닫아야 본사 폐업 가능
- 자동 생성 LB 서비스는 Service(type LoadBalancer) 삭제와 함께 정리되는지 확인 후 LB 삭제 → 남은 자원은 비용 발생

20. 운영 체크리스트와 비유로 다시 보기

SKE 운영 체크리스트 (SCP 매뉴얼 + 교안 + 실무)

분류	확인 항목	이유
포트	UDP 4789(calico-vxlan), TCP 5473(calico-typha), 10250(kubelet), 19100(node-exporter), 19400(dcgm-exporter)은 서비스에 사용 금지·차단 금지	SKE 관리용. 막으면 노드 기능 장애
OS 방화벽	Calico 사용 노드에는 OS 방화벽 미사용 권장	Pod 네트워크 차단
접근	API Server는 프라이빗 엔드포인트 + Bastion, 제어 영역 로깅 사용	관리 경로 최소화, 감사 추적
버전	kubectl ±1 마이너, EoTS 일정 확인	EoTS 후 신규 생성 불가, 취약점
노드	노드 수 조정 시 Pod 이동 → 작업 시간대 조정, 고스펙(GPU 등) 증설 전 SCP에 자원 여유 확인	서비스 순단, 증설 부분 실패
자동 복구	Creating에서 멈춘 노드나 한 풀에서 동시 다수 장애는 자동 복구 불가 (매뉴얼)	자동 복구만 믿지 않기
LB	자동 생성 LB는 annotation으로만 변경	Reconcile 원복
워크로드	Pod 2개 이상 + readinessProbe + requests 설정	노드 작업·배포 중 무중단
비용	삭제는 Apps → 노드 풀 → 클러스터 → File Storage·LB 순	남은 VM·NAS·LB 과금

비유로 다시 보기 - 장애 상황 대응

상황	프랜차이즈로 보면	쿠버네티스 동작 / 확인 지점
Pod가 계속 재시작	직원이 출근하자마자 쓰러짐	CrashLoopBackOff → `kubectl logs`, liveness 설정
Pod가 Pending	배치할 매장에 자리 없음	노드 자원·requests·taint → `kubectl describe pod` Events
404	안내 직원은 있는데 안내판에 해당 매장이 없음	Ingress Host·Path, 접속 도메인
502	안내 직원이 매장에 전화했는데 아무도 안 받음	Service selector·targetPort, Pod readiness
콘솔 수정이 되돌아감	감독관이 지침서대로 원상 복구	정답지(YAML·annotation)를 수정
kubectl 접속 불가	출입증이 없거나 출입 명단에 없음	kubeconfig, 프라이빗 엔드포인트 허용 VM, 6443 SG 규칙

정리하며 얻은 인사이트

- 쿠버네티스를 한 문장으로 줄이면 "원하는 상태를 적어 두면 시스템이 계속 맞춰 주는 구조". 자동 복구도 롤링 업데이트도 LB 원복도 모두 이 한 가지 원리에서 나옴
- SCP Kubernetes Engine은 본사(Control Plane)만 대신 운영해 줄 뿐, 노드·네트워크·LB·스토리지는 9월에 정리한 IaaS 원칙(최소 노출, 경로 분리, 장애 전제)이 그대로 적용
- 교안 버전(v1.24·v1.26)은 현재 생성 불가 → 화면·버전은 최신 콘솔과 매뉴얼로 다시 확인 필요